

7 Advanced RAG Patterns

A Field Guide

Query Transformation · Hybrid Search · Re-Ranking · Hierarchical Retrieval
Corrective RAG · Agentic RAG · Graph RAG

Practical patterns for production-grade retrieval-augmented generation

Table of Contents

01	Pattern 1 — Query Transformation	p.04
02	Pattern 2 — Hybrid Search	p.07
03	Pattern 3 — Re-Ranking	p.10
04	Pattern 4 — Hierarchical Retrieval	p.13
05	Pattern 5 — Corrective RAG (CRAG)	p.16
06	Pattern 6 — Agentic RAG	p.19
07	Pattern 7 — Graph RAG	p.22

Introduction

Naive RAG — embed, retrieve top-k, stuff into a prompt — breaks down fast in production. Vocabulary mismatches, noisy chunks, missing structure, and single-hop retrieval all cap how far a basic pipeline can go. This guide walks through seven patterns that address those failure modes directly, each with an architecture diagram, a data-flow breakdown, and clear guidance on when to reach for it.

How to use this guide

- Each pattern stands alone — adopt one or combine several as your pipeline matures.
- Diagrams favor clarity over completeness; adapt architecture to your own stack.
- The comparison table at the end helps you pick a starting point fast.

1. Query Transformation

Close the vocabulary gap between how users ask and how documents are written.

Overview

Users rarely phrase questions the way source documents phrase answers. Query transformation rewrites, expands, or reframes the incoming query before retrieval ever happens — using an LLM to generate better search inputs instead of hoping the raw user text embeds close enough to the right chunk.

How it works

- Take the raw user query as input to a lightweight LLM call
- Generate multiple variants: paraphrases, sub-questions, or a hypothetical answer (HyDE)
- Optionally decompose complex queries into simpler sub-queries
- Retrieve using each variant in parallel against the vector store
- Merge/de-duplicate the combined candidate set before ranking

✓ Pros

- Fixes vocabulary & phrasing mismatch directly at the source
- HyDE especially strong when queries are short and documents are long-form
- Decomposition unlocks multi-part questions naive RAG can't handle

✗ Cons

- Adds one extra LLM call (latency + cost) before retrieval even starts
- Poorly tuned rewrites can drift away from user intent
- Multiple variants means more retrieval calls to fan-in and merge

USE WHEN

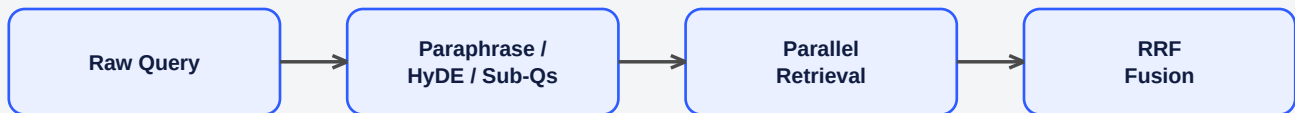
Users type short, casual, or vague queries against long, formal technical documents.

Query Transformation — Architecture



The rewriter sits between the user and the vector store. HyDE's trick: instead of embedding the question, it asks the LLM to write a fake answer, then embeds THAT — because answers look more like the documents than questions do.

Query Transformation — Data Flow



Reciprocal Rank Fusion (RRF) combines ranked lists from each query variant without needing comparable similarity scores — a document ranked highly by any variant rises to the top.

2. Hybrid Search

Combine dense embeddings with sparse keyword search — neither alone is enough.

Overview

Dense vector search excels at semantic similarity but can miss exact keyword matches (product codes, names, acronyms). Sparse methods like BM25 excel at exact term matching but miss paraphrases and synonyms. Hybrid search runs both in parallel and fuses the results, capturing the strengths of each.

How it works

- Index documents twice: once as dense embeddings, once as a sparse (BM25/TF-IDF) index
- At query time, run the same query through both retrieval paths
- Each path returns its own ranked candidate list with different scoring scales
- Fuse the two ranked lists using Reciprocal Rank Fusion (RRF) or a weighted score
- Pass the fused, deduplicated top-k forward to the next pipeline stage

✓ Pros

- Catches exact matches (IDs, codes, rare terms) that embeddings blur over
- Catches paraphrases and synonyms that keyword search misses entirely
- RRF fusion needs no score calibration between the two systems

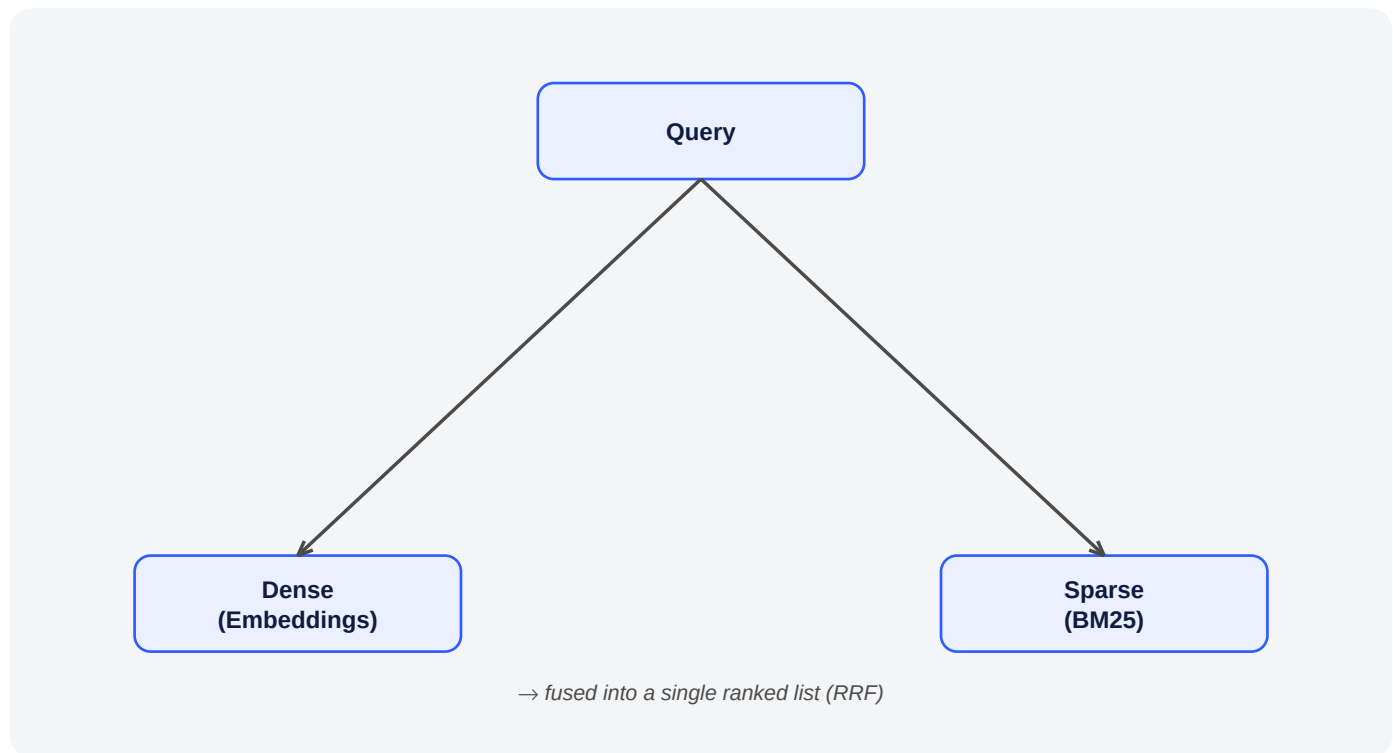
✗ Cons

- Requires maintaining two indexes instead of one
- Slightly higher infra complexity and query latency
- Fusion weighting may need tuning per domain

USE WHEN

Your corpus mixes natural language with structured terms — product SKUs, legal citations, medical codes, proper nouns.

Hybrid Search — Architecture



Query goes down two independent paths simultaneously — dense (embedding cosine similarity) and sparse (BM25 keyword overlap) — then both candidate sets are merged by a fusion step.

Hybrid Search — Data Flow



RRF scores each document by $1/(k+\text{rank})$ summed across both retrievers ($k \approx 60$ is a common default). A document doesn't need to be top-1 in either list — being decent in both is enough to win.

3. Re-Ranking

Trade a little latency for a lot of precision using a second, smarter model pass.

Overview

Initial retrieval (bi-encoder / vector search) is optimized for speed across millions of documents, not for pinpoint accuracy. Re-ranking adds a second stage: a slower but far more accurate cross-encoder model re-scores a small candidate set — typically the top 20-50 — before the final top-k reaches the LLM.

How it works

- Retrieve a wide candidate set (e.g. top-50) cheaply with the standard vector index
- Pass each (query, candidate) pair jointly into a cross-encoder model
- The cross-encoder outputs a fine-grained relevance score per pair
- Sort candidates by this new score and keep only the true top-k (e.g. top-5)
- Forward only the highest-precision subset into the LLM's context window

✓ Pros

- Meaningfully improves precision of what actually reaches the LLM
- Reduces irrelevant context — lowers hallucination risk from noisy chunks
- Works as a drop-in layer on top of any existing retriever

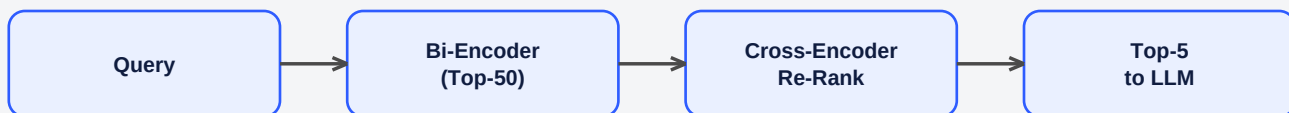
✗ Cons

- Cross-encoders are far slower — cannot score millions of docs directly
- Adds a model hop, increasing end-to-end latency
- Needs a hosted or self-served re-ranker model (e.g. Cohere Rerank, BGE-reranker)

USE WHEN

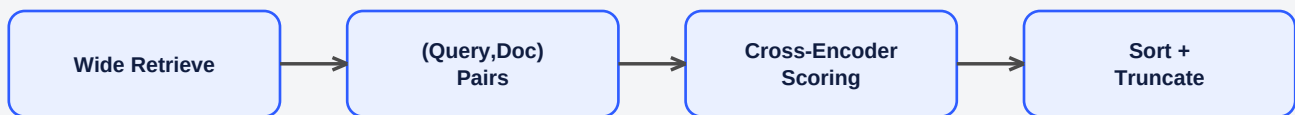
Retrieval recall is fine but generation quality suffers from noisy, loosely-relevant context.

Re-Ranking — Architecture



A funnel pattern: cast a wide, cheap net with the bi-encoder, then spend expensive compute only on the shortlist via the cross-encoder before anything reaches the LLM.

Re-Ranking — Data Flow



Bi-encoders embed query and document independently (fast, cacheable). Cross-encoders read both together with full attention (slow, but understands nuance bi-encoders miss).

4. Hierarchical Retrieval

Search small, precise chunks — but hand the LLM the full surrounding context.

Overview

Small chunks embed and match precisely but strip away surrounding context. Large chunks preserve context but embed poorly and dilute relevance. Small-to-big (parent-child) retrieval resolves this tension: index small child chunks for matching, but return their larger parent chunk for generation.

How it works

- Split each document into large 'parent' chunks (e.g. ~2000 tokens)
- Split each parent further into small 'child' chunks (e.g. ~200 tokens)
- Embed and index only the child chunks for precise similarity search
- At query time, retrieve matching child chunks as usual
- Before generation, swap each matched child for its full parent chunk

✓ Pros

- Retrieval precision of small chunks, context completeness of large chunks
- Reduces 'lost context' failures on nuanced or multi-sentence answers
- Works well for long structured documents: manuals, contracts, textbooks

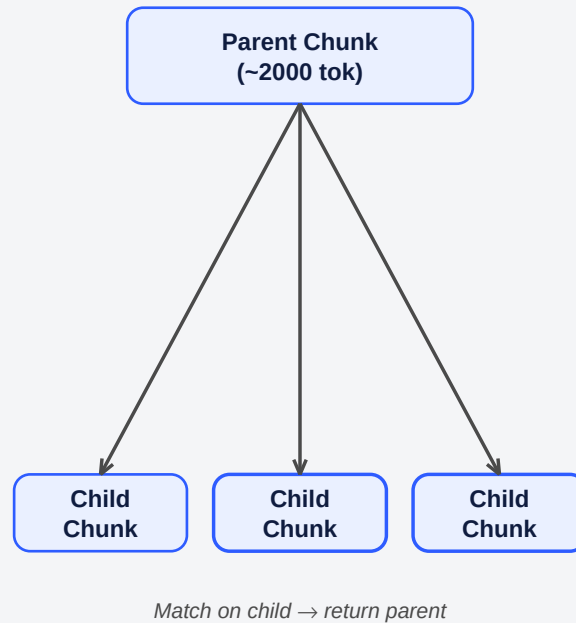
✗ Cons

- Requires maintaining a parent-child mapping alongside the vector index
- Parent chunks consume more context window tokens per result
- Slightly more complex ingestion pipeline to build and version

USE WHEN

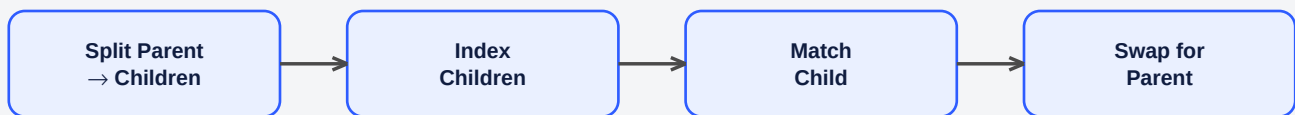
Source documents are long and structured, and answers depend on surrounding context, not isolated sentences.

Hierarchical Retrieval — Architecture



Parent chunks are split into small children for indexing. A query matching a single small child still returns the entire parent chunk — precision in, context out.

Hierarchical Retrieval — Data Flow



Flat chunking forces a trade-off between match precision and context completeness. Small-to-big retrieval gets both by decoupling what's searched from what's returned.

5. Corrective RAG (CRAG)

Never blindly trust the retriever — grade results and self-correct in real time.

Overview

Standard RAG assumes retrieved chunks are relevant. Corrective RAG adds a lightweight grading step: an LLM (or classifier) scores each retrieved chunk for relevance before generation. When retrieval quality is poor, the system triggers a fallback — commonly a live web search — rather than generating from bad context.

How it works

- Run standard retrieval to get an initial candidate set
- Pass each candidate through a lightweight relevance grader (LLM-as-judge)
- Classify results as Correct, Ambiguous, or Incorrect
- If results are weak, trigger a corrective action: query rewrite + web search
- Combine validated internal context with any corrective results, then generate

✓ Pros

- Catches and recovers from bad retrieval instead of silently hallucinating
- Web-search fallback keeps answers current for time-sensitive queries
- Grading step is model-agnostic — works with any base retriever

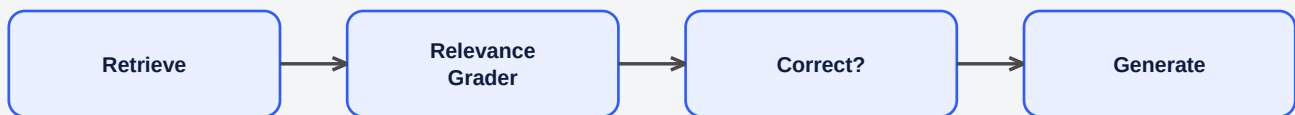
✗ Cons

- Adds a grading LLM call on every request — extra latency and cost
- Fallback web search introduces external dependency and variability
- Grader itself can be wrong, adding another point of failure

USE WHEN

Answer correctness matters more than latency — high-stakes or public-facing assistants.

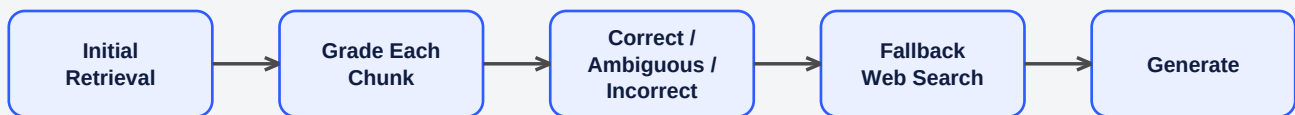
Corrective RAG (CRAG) — Architecture



If Incorrect → Rewrite + Web Search fallback

Every retrieved chunk passes through a relevance grader. Relevant chunks flow straight to generation; irrelevant ones trigger a corrective fallback path before generation happens.

Corrective RAG (CRAG) — Data Flow



The grading loop turns retrieval from a one-shot gamble into a checked, correctable step — the system knows when it doesn't know, rather than confidently generating from noise.

6. Agentic RAG

Give the LLM a router and a toolbox instead of one fixed retrieval path.

Overview

Not every question is answerable from a single vector search. Agentic RAG wraps retrieval inside an agent loop: a planning/router step decides which tool(s) to call — vector search, SQL, an API, a calculator, live web search — and can chain multiple calls together to answer multi-hop, multi-source questions.

How it works

- A router/planner agent receives the query and decides an execution plan
- Complex queries get decomposed into an ordered sequence of tool calls
- Each tool (retriever, SQL, API, calculator) executes and returns structured results
- The agent inspects intermediate results and may re-plan or call more tools
- Once enough grounded evidence is gathered, the agent synthesizes a final answer

✓ Pros

- Handles multi-hop questions spanning multiple data sources
- Extends RAG beyond static documents into live APIs and structured data
- Self-correcting — agent can retry or choose a different tool if one fails

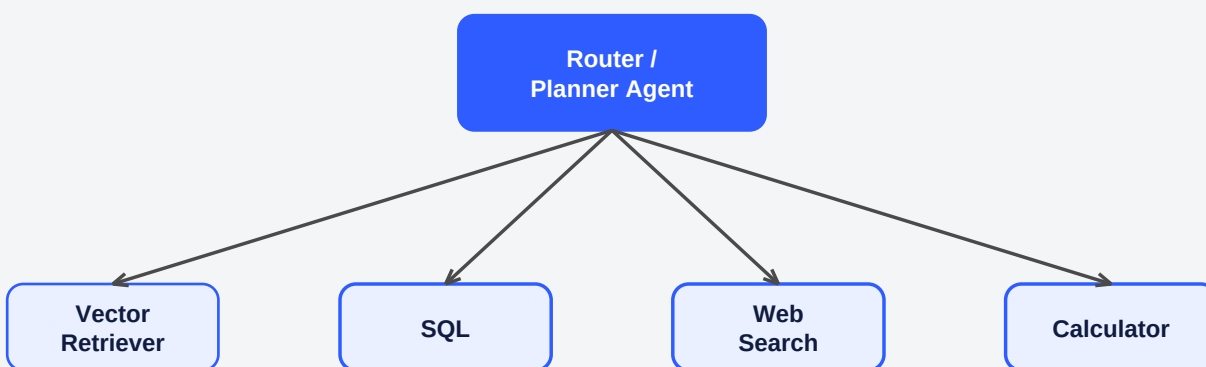
✗ Cons

- Highest latency and cost of any pattern in this guide — multiple LLM + tool round-trips
- Harder to debug and evaluate — non-deterministic execution paths
- Requires careful guardrails to prevent runaway tool-call loops

USE WHEN

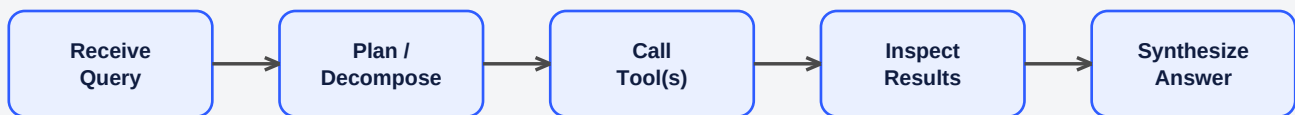
Questions genuinely require combining multiple data sources or real-time information, not just document lookup.

Agentic RAG — Architecture



A router agent sits at the center, dispatching to whichever tools the query actually needs — vector retriever, SQL, web search, or a calculator — then synthesizes across all results.

Agentic RAG — Data Flow



Complex questions get decomposed into ordered sub-tasks, each handled by the right tool, with the agent chaining outputs from one step into the input of the next.

7. Graph RAG

Reason over explicit relationships, not just semantic proximity.

Overview

Vector similarity cannot answer relationship questions — 'who reports to whom', 'which products depend on which suppliers'. Graph RAG extracts entities and relationships from source documents into a knowledge graph, then answers queries by traversing that graph rather than (or alongside) vector similarity search.

How it works

- Run an LLM extraction pass over documents to identify entities and relationships
- Build a knowledge graph: nodes are entities, edges are typed relationships
- At query time, identify the relevant entity/entities mentioned in the query
- Traverse relevant edges to gather connected facts (1-hop, 2-hop, etc.)
- Combine traversed graph facts with any vector-retrieved text, then generate

✓ Pros

- Answers multi-hop, relationship-based questions vector search structurally cannot
- Produces explainable, traceable reasoning paths through explicit graph edges
- Can combine with vector search for hybrid graph + semantic retrieval

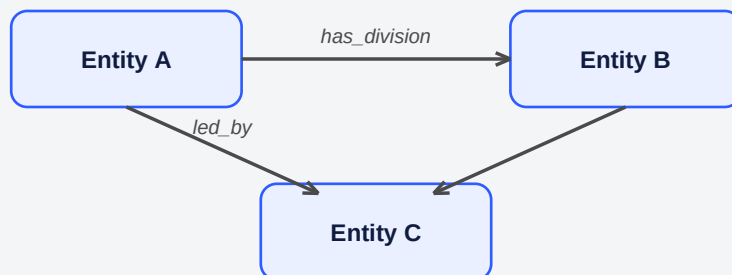
✗ Cons

- Graph construction (entity/relation extraction) is expensive and error-prone at scale
- Requires ongoing graph maintenance as source documents change
- Higher engineering overhead than a standard vector index

USE WHEN

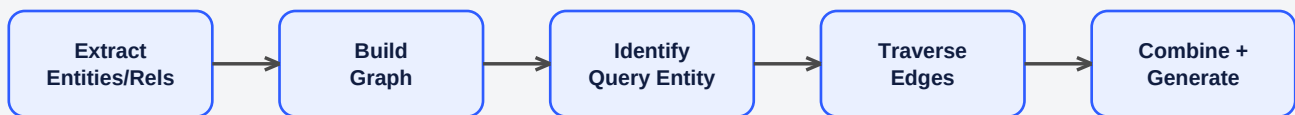
Your domain is relationship-rich — org charts, supply chains, financial filings, scientific citations.

Graph RAG — Architecture



Documents are processed once to extract entities and relationships into a graph structure that can be traversed at query time — independent of surface-level text similarity.

Graph RAG — Data Flow



Answering 'who leads Acme's AI division' means walking two typed edges — 'has_division' then 'led_by' — a hop-by-hop reasoning chain no embedding similarity score can replicate.

Pattern Comparison

Pattern	Fixes	Cost/Latency	Best Fit
Query Transform	Vocabulary gap	Low-Med	Short/casual queries
Hybrid Search	Missed keywords	Low	Mixed NL + codes/IDs
Re-Ranking	Noisy top-k	Medium	Precision-sensitive Q&A
Hierarchical	Context loss	Low	Long structured docs
Corrective RAG	Bad retrieval	Medium	High-stakes accuracy
Agentic RAG	Single-hop limit	High	Multi-source questions
Graph RAG	No relationships	Med-High	Relationship-rich domains

**Choose the pattern that fits your failure mode —
not the most sophisticated one.**

End of Field Guide